

PROOF OF CONCEPT

Banking Regulatory Compliance Copilot

An Enterprise RAG-Based AI Assistant for Regulatory Interpretation, Policy Mapping & Audit Readiness

Document Code: POC-BANK-002

Classification: Internal — AI Mentor Review

Prepared by: AI Solutions Architecture Team

Version 1.0 — POC Approval Draft

Table of Contents

1. Executive Summary
2. Problem Statement
3. Proposed AI Solution
4. System Architecture
5. Complete Folder Structure
6. Functional Modules
7. APIs
8. Database Design
9. RAG Pipeline
10. LangChain + LangGraph Flow
11. Tech Stack
12. Production Features
13. Future Enhancements
14. Risks
15. Development Roadmap
16. Conclusion

1. Executive Summary

1.1 The Business Problem

Compliance teams at banks and financial institutions must continuously interpret and apply regulations issued by central banks, prudential regulators, and international standard-setting bodies against thousands of pages of internal policies, procedures, and product documentation. Determining whether a specific product, process, or transaction complies with current regulation requires manually cross-referencing regulatory text against internal policy, a process that does not scale with the pace of regulatory change.

1.2 Why This Problem Persists

Regulatory bodies issue guidance, circulars, and amendments on a near-continuous basis, often with overlapping or superseding provisions, while internal policies are updated on a separate, slower cadence. The two document sets live in different systems, are maintained by different teams, and are rarely cross-linked, so gaps between 'what the regulator now requires' and 'what our policy currently says' can persist undetected for months.

1.3 Why Current Solutions Fall Short

Manual regulatory-change tracking via spreadsheets and email distribution lists cannot keep pace with the volume of issuances, and generic keyword search across regulatory portals returns whole documents rather than the specific clause relevant to a given internal control, forcing compliance officers to read extensively to confirm coverage.

1.4 Why AI + Retrieval-Augmented Generation (RAG)

RAG allows the assistant to retrieve the exact, currently effective regulatory clause alongside the corresponding internal policy section, enabling gap identification and grounded answers rather than generic regulatory summaries that may not reflect the institution's specific policy language or jurisdiction.

1.5 Business Value

- Accelerates regulatory-change impact assessment from weeks to days.
- Surfaces gaps between current regulation and internal policy before they become audit findings.
- Provides examiners and auditors with a traceable, citation-backed compliance narrative.
- Reduces reliance on outside counsel for routine regulatory interpretation questions.
- Improves consistency of compliance guidance given to front-line business units.

1.6 Target Users

Persona	Role	Primary Need
Compliance Officer	Interprets regulation and maps it to policy	Fast, cited regulatory answers
Internal Auditor	Tests control effectiveness against regulation	Traceable evidence linking control to rule
Legal Counsel	Reviews high-risk interpretations	Confidence-scored answers with source citations
Risk Manager	Assesses regulatory exposure	Analytics on emerging regulatory themes

1.7 Expected Impact

- Target: 50%+ reduction in time to assess the impact of a new regulatory issuance.
- Target: measurable reduction in policy-regulation gaps identified during audit vs. proactively.
- Consistent, citation-backed compliance guidance across business units.
- Improved examiner confidence through demonstrable audit trails.

2. Problem Statement

2.1 Detailed Problem Description

Banks operate under overlapping regulatory regimes — prudential capital rules, conduct regulations, anti-money-laundering requirements, and data protection law — each with its own issuing body, update cadence, and document format. Compliance teams must maintain a continuously accurate mapping between this external regulatory corpus and thousands of pages of internal policy, procedure, and product documentation, a task performed today largely through manual review.

2.2 Industry Examples

Organization Type	Scenario	Consequence
Regional bank	A prudential capital treatment change is issued but not cross-referenced against the internal capital policy for six months.	Policy misalignment flagged as a material finding during the next regulatory examination.
Global bank subsidiary	Conduct regulation update affecting product disclosures is missed because it was published in a bulletin the compliance team does not routinely monitor.	Remediation program required after a customer complaint pattern surfaces the gap.
Community bank	New AML guidance is manually summarized inconsistently by two analysts, producing conflicting internal guidance.	Front-line staff apply inconsistent enhanced due diligence thresholds.

2.3 Current Workflow (As-Is)

1. Regulatory bulletin or circular is published by the relevant authority.
2. A compliance analyst is assigned to read and summarize the issuance.
3. Analyst manually searches internal policy repositories for potentially affected sections.
4. Analyst drafts an impact memo and routes it for legal and management review.
5. Policy owners are notified to update affected documents, on a manual, email-driven basis.
6. Completion of policy updates is tracked in a spreadsheet with no systematic verification.

2.4 Pain Points

- No automated linkage between external regulatory text and internal policy sections.
- High dependency on individual analysts correctly identifying all affected policies.
- Long lag between regulatory issuance and confirmed internal policy alignment.
- Inconsistent interpretation memos produced by different analysts for similar issuances.
- Audit evidence of compliance is assembled manually and inconsistently across examinations.

2.5 Compliance, Regulatory & Documentation Challenges

Prudential regulators and examiners expect institutions to demonstrate, with evidence, that internal policy reflects currently effective regulation and that gaps are identified and remediated in a timely manner. Failure to

demonstrate this traceability is a recurring source of examination findings, remediation programs, and in serious cases, formal enforcement action.

3. Proposed AI Solution

3.1 Solution Overview

The Banking Regulatory Compliance Copilot ingests both the external regulatory corpus and the institution's internal policy library, indexes them into a shared, cross-linked knowledge base, and allows compliance staff to ask natural-language questions that are answered with grounded, dual-sourced citations — one to the regulation, one to the corresponding internal policy — with any detected misalignment explicitly surfaced.

3.2 Why Retrieval-Augmented Generation Is the Correct Approach

Regulatory interpretation must be grounded in the exact, currently effective clause; a model's general training knowledge cannot reliably reflect jurisdiction-specific, frequently amended banking regulation, so retrieval against a maintained, versioned corpus is required for any answer to be examiner-defensible.

3.3 Why a Standard Chatbot Is Not Sufficient

A standard chatbot cannot compare a bank's specific internal policy wording against current regulation, and any hallucinated regulatory citation in this domain carries direct examination and enforcement risk, making ungrounded generation unacceptable.

3.4 Why Vector Search Is Required

Compliance questions are often framed around a business scenario ('can we onboard this customer type without enhanced due diligence') rather than the precise regulatory clause number, so semantic retrieval is required to connect business language to the applicable rule, complemented by exact-match search for citation-number lookups.

3.5 How the LLM Improves the Workflow

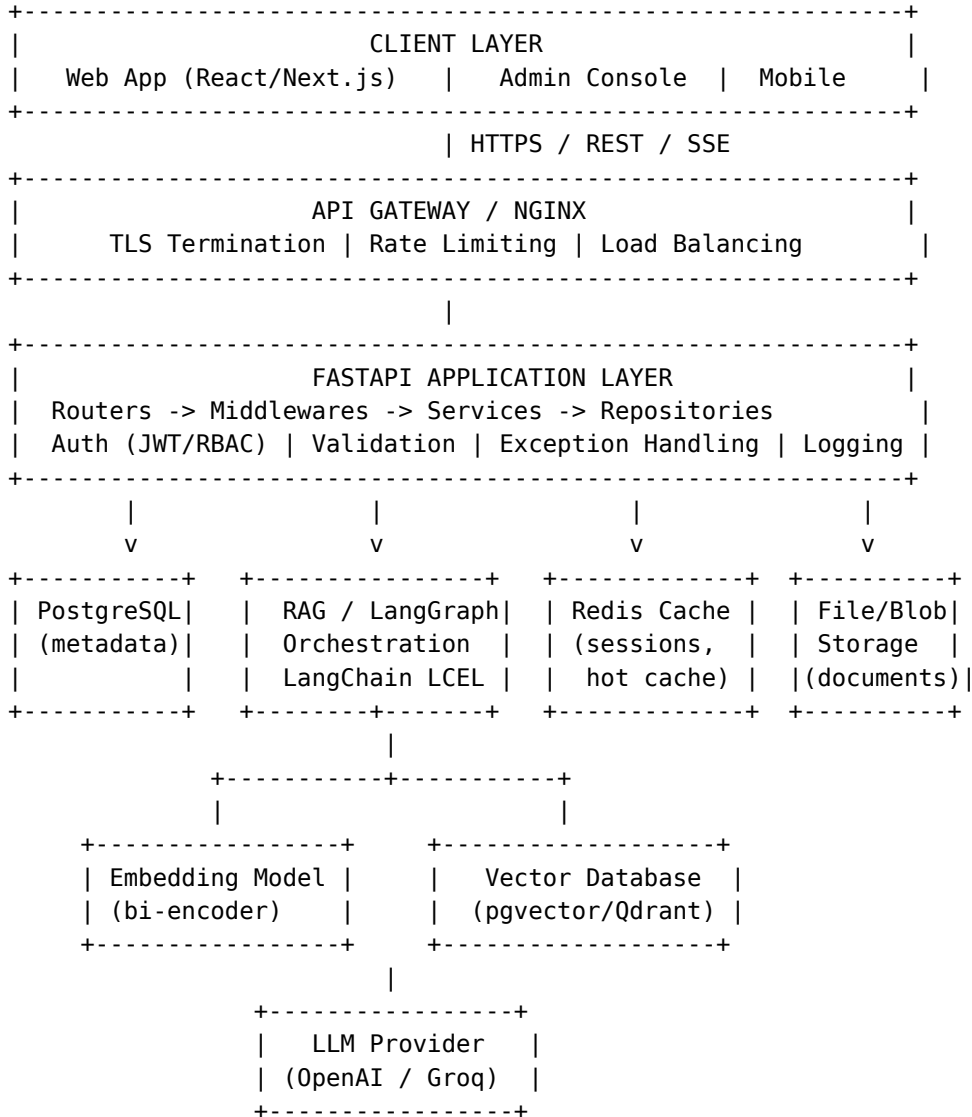
The LLM synthesizes matched regulatory and policy passages into a coherent gap analysis, explicitly stating where policy aligns, where it appears to diverge, and where the regulation itself is ambiguous, giving compliance officers a structured starting point rather than raw source text to reconcile manually.

4. System Architecture

4.1 Architectural Overview

The Banking Regulatory Compliance Copilot platform is built as a modular, cloud-deployable, layered architecture. Each layer has a single responsibility, communicates through well-defined contracts, and can be scaled or replaced independently. This separation of concerns is essential for an enterprise system that must evolve as banking regulatory compliance requirements, models, and regulations change over time.

4.2 High-Level Component Diagram



4.3 Component Responsibilities

Component	Responsibility
Frontend	Chat UI, document upload, admin dashboards, citation rendering, streaming response display.
API Gateway / Nginx	TLS termination, reverse proxy, load balancing, request throttling.

Component	Responsibility
FastAPI Backend	Async REST API layer; routes requests to services; enforces auth and validation.
Authentication (JWT)	Issues and verifies signed access/refresh tokens; short-lived access, rotating refresh.
Role-Based Access Control	Maps users to roles (Admin, Reviewer, Analyst, Viewer) and enforces per-endpoint scopes.
LangChain (LCEL)	Declarative composition of prompt -> retriever -> LLM -> parser as runnables.
LangGraph	Stateful orchestration: conditional routing, retries, human-approval nodes, multi-step agents.
Document Parser	Extracts text/tables from PDF, DOCX, TXT, HTML while preserving structure and page numbers.
Chunking Engine	Splits parsed text into semantically coherent chunks with overlap and metadata tags.
Embedding Model	Converts chunks and queries into dense vectors for semantic similarity search.
Vector Database	Stores embeddings and metadata; supports ANN search and hybrid (keyword + vector) queries.
Retriever	Fetches top-k relevant chunks per query, applies metadata filters (date, doc type, role).
Reranker	Cross-encoder re-scoring of retrieved chunks to improve precision before context injection.
Prompt Template	Structured, versioned prompts with system instructions, guardrails, and citation formatting.
Output Parser	Validates and structures LLM output (JSON schema, citation extraction, refusal detection).
LLM	Generates grounded natural-language answers strictly from the retrieved context.
PostgreSQL	System of record for users, roles, documents, conversations, audit logs, analytics.
Redis	Session cache, rate-limit counters, short-term conversation memory, embedding cache.
File / Blob Storage	Durable storage of original source documents and processed artifacts.
Logging & Monitoring	Structured logs, metrics, tracing across every request for observability and audit.
CI/CD	Automated build, test, container image publishing, and deployment pipeline.

4.4 End-to-End Data Flow

The following sequence describes how a single user query flows through the system, from ingestion of source documents to the delivery of a grounded, cited answer:

7. Documents are uploaded via the Document Upload API and stored in Blob Storage.
8. The Document Parser extracts raw text and structural metadata (page, section, table).

9. The Chunking Engine splits content into overlapping chunks and tags each with source metadata.
10. The Embedding Model converts each chunk into a vector, which is written to the Vector Database alongside metadata.
11. A user submits a natural-language question through the Chat API.
12. The query is embedded and passed to the Retriever, which performs a hybrid (vector + keyword) search filtered by the user's role and permitted document scope.
13. The Reranker re-orders the top candidate chunks by relevance to the exact query.
14. LangChain (LCEL) assembles a Prompt Template containing the system instructions, the reranked context, and conversation memory.
15. LangGraph routes the request: if confidence is low or the query touches a sensitive workflow, it is routed to a human-approval node before finalization.
16. The LLM generates a grounded response with inline citations back to source chunks.
17. The Output Parser validates the response structure and citation integrity before it is returned to the client.
18. The interaction, retrieved sources, and response are persisted to PostgreSQL for audit and analytics; the response is streamed to the client over SSE.

5. Complete Folder Structure

5.1 Repository Layout

```

src/
  app/
    routers/          # FastAPI route definitions grouped by domain (auth, chat, documents,
admin)                # Business logic orchestrating repositories, RAG pipeline, and external
  services/           # Data access layer – encapsulates all PostgreSQL/ORM queries
  calls               # DB session management, connection pooling, Alembic migrations
  repositories/       # SQLAlchemy ORM models (User, Document, Conversation, AuditLog, etc.)
  database/           # Pydantic request/response schemas (DTOs) with validation rules
  models/             # Auth, logging, rate-limiting, correlation-id, exception-handling
  schemas/            # Environment-based settings (Pydantic Settings), secrets loading
  middlewares/        # Security utilities (JWT, hashing), constants, dependency injection
middleware            # Generic helpers (pagination, formatting, retry decorators)
  config/             # Versioned prompt templates (system, retrieval, citation,
  core/               # Embedding client wrapper, batching, caching logic
wiring                # Vector DB client, index management, hybrid-search implementation
  utils/              # Parsers for PDF/DOCX/TXT, chunking strategies
  prompts/            # Retriever, reranker, context-builder, citation-formatter
  summarization)      # StateGraph definitions, node functions, conditional edges, agent
  embeddings/         # Unit, integration, and end-to-end tests (pytest)
  vectorstore/        # Local log output (structured JSON, rotated)
  documents/          # FastAPI application entrypoint / ASGI app factory
  rag/                # Database migration scripts
  langgraph/          # Dockerfile(s), docker-compose for local/dev stacks
flows                 # CI/CD pipeline definitions (GitHub Actions)
  tests/              # React/Next.js client application
  logs/               # Chat window, citation cards, upload widget, admin tables
  main.py             # Route-level pages
alembic/              # Data-fetching and auth hooks
docker/               # Documented environment variable template
.github/workflows/    # Python dependency and tooling configuration
frontend/             # Architecture overview and setup instructions
  src/components/
  src/pages/
  src/hooks/
.env.example
pyproject.toml
README.md

```

5.2 Purpose of Each Folder

Folder	Purpose
routers/	Thin HTTP layer; parses requests, calls services, returns schemas. No business logic.
services/	Coordinates use cases (e.g. 'ask a question', 'ingest a document'); the core business logic.
repositories/	Isolates all SQL/ORM access so services never talk to the database directly.
database/	Engine/session setup, connection pooling, and Alembic migration configuration.
models/	Declarative ORM entities mapped to PostgreSQL tables.

Folder	Purpose
schemas/	Pydantic models defining and validating API request/response contracts.
middlewares/	Cross-cutting concerns applied to every request: auth, logging, rate limits.
config/	Centralized, environment-aware configuration (dev/staging/prod).
core/	Security primitives, dependency injection, shared constants.
utils/	Reusable helper functions not tied to a specific domain.
prompts/	Prompt templates kept out of code, versioned, and independently testable.
embeddings/	Wraps the embedding provider; adds batching, retries, and caching.
vectorstore/	Abstracts the vector database so the provider can be swapped with minimal change.
documents/	Format-specific parsing and chunking logic.
rag/	The retrieval-augmented generation pipeline: retrieve, rerank, build context, cite.
langgraph/	Stateful multi-step orchestration graphs and node implementations.
tests/	Automated test suite covering unit, integration, and RAG-quality evaluation.

6. Functional Modules

The platform is decomposed into the following functional modules. Each module is independently testable and maps to a bounded context within the banking regulatory compliance domain.

Authentication

SSO-integrated login with JWT session tokens for compliance, audit, and legal staff.

User Management

Provisioning with business-unit and regulatory-domain scoped access.

Admin Dashboard

Corpus freshness monitoring across regulatory sources and policy repositories.

Document Upload

Ingestion of regulatory issuances and internal policy documents in PDF, DOCX, TXT.

Document Processing

Structure-aware parsing with clause-level segmentation and effective-date tagging.

Embedding Pipeline

Vectorization of new regulatory and policy content, cross-linked at ingestion time.

Vector Database

Dual-indexed store distinguishing regulatory-source vectors from internal-policy vectors.

Chat Module

Conversational Q&A returning paired regulation-and-policy citations with gap flags.

Conversation Memory

Retains context across a multi-turn regulatory impact assessment session.

Search

Direct hybrid search across regulation and policy corpora independent of chat.

Audit Logs

Immutable record supporting examiner review of every AI-assisted determination.

Notifications

Alerts policy owners when a newly ingested regulation maps to their policy area.

Analytics

Dashboards on regulatory-change volume, gap trends, and resolution timelines.

Reports

Exportable gap-analysis and impact-assessment reports for examiner packages.

Settings

Regulatory-domain subscriptions and jurisdiction configuration per business unit.

Role Management

RBAC distinguishing Compliance Officer, Auditor, Legal, and Risk Manager scopes.

7. APIs

7.1 API Design Principles

- RESTful resource-oriented URLs (e.g. /api/v1/documents/{id}).
- Consistent envelope for success and error responses.
- Cursor and offset pagination supported on all list endpoints via ?page & ?page_size.
- Filtering via query parameters (?status=, ?doc_type=, ?date_from=, ?date_to=).
- Sorting via ?sort_by= and ?order=asc|desc.
- Idempotent PUT/DELETE; partial updates via PATCH where applicable.
- All endpoints versioned under /api/v1/ to allow non-breaking evolution.

7.2 Core Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/login	Authenticate user, issue JWT access + refresh tokens.
POST	/api/v1/auth/refresh	Exchange a valid refresh token for a new access token.
GET	/api/v1/users/me	Return the authenticated user's profile and role.
POST	/api/v1/documents	Upload a new source document for ingestion.
GET	/api/v1/documents	List documents with pagination, filtering, and sorting.
GET	/api/v1/documents/{id}	Retrieve a single document's metadata and processing status.
DELETE	/api/v1/documents/{id}	Soft-delete a document and its vector embeddings.
POST	/api/v1/chat/sessions	Create a new conversation session.
POST	/api/v1/chat/sessions/{id}/messages	Submit a question; returns a streamed, cited answer.
GET	/api/v1/chat/sessions/{id}	Retrieve full conversation history.
GET	/api/v1/audit-logs	List audit log entries (Admin only), paginated and filterable.
GET	/api/v1/analytics/usage	Aggregate usage metrics for dashboards.

7.3 Standard Response Envelope

```
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "page_size": 20, "total_items": 134, "total_pages": 7 },
  "error": null
}
```

7.4 Standard Error Response

```
{
  "success": false,
  "data": null,
  "meta": null,
  "error": {
    "code": 400,
    "message": "Invalid request"
  }
}
```

```

"error": {
  "code": "DOCUMENT_NOT_FOUND",
  "message": "No document exists with the given ID.",
  "request_id": "3f9a2c11-..."
}
}

```

7.5 Sample Request / Response — Ask a Question

Request:

POST /api/v1/chat/sessions/8f21/messages

Authorization: Bearer <jwt>

Content-Type: application/json

```

{
  "message": "Does our current KYC policy meet the enhanced due diligence threshold introduced in the latest AML guidance?"
}

```

Response:

```

{
  "success": true,
  "data": {
    "message_id": "9c31-...",
    "answer": "The current KYC policy sets the enhanced due diligence trigger at a higher risk threshold than the level specified in the latest AML guidance; policy Section 4.2 will require an update to align with the revised threshold.",
    "citations": [
      { "source": "Internal_KYC_Policy_v7.docx", "page": 4 }
    ],
    "confidence": "high"
  },
  "error": null
}

```

7.6 HTTP Status Codes Used

Code	Meaning	Usage
200	OK	Successful GET/PUT/PATCH.
201	Created	Successful POST creating a resource.
204	No Content	Successful DELETE.
400	Bad Request	Validation failure on request payload.
401	Unauthorized	Missing or invalid JWT.
403	Forbidden	Authenticated but insufficient role/permission.
404	Not Found	Resource does not exist.
409	Conflict	Duplicate resource or state conflict.
422	Unprocessable Entity	Semantically invalid request (e.g. unsupported file type).
429	Too Many Requests	Rate limit exceeded.
500	Internal Server Error	Unhandled server-side exception.

8. Database Design

8.1 Core Tables

Table	Key Columns	Notes
users	id (PK), email, hashed_password, role_id (FK), created_at	One row per authenticated user.
roles	id (PK), name, permissions (JSONB)	Defines RBAC permission sets.
documents	id (PK), title, doc_type, storage_uri, status, uploaded_by (FK users), created_at	Metadata for every ingested source document.
document_chunks	id (PK), document_id (FK), chunk_index, content, token_count, page_number	Chunk-level metadata; vectors live in the vector store, keyed by chunk id.
conversations	id (PK), user_id (FK), title, created_at	One row per chat session.
messages	id (PK), conversation_id (FK), role, content, created_at	Individual turns within a conversation.
citations	id (PK), message_id (FK), document_chunk_id (FK), relevance_score	Links an answer to the exact source chunks used.
audit_logs	id (PK), user_id (FK), action, resource_type, resource_id, ip_address, created_at	Immutable record of security-relevant actions.
notifications	id (PK), user_id (FK), type, payload (JSONB), read_at	In-app notification queue.

8.2 Relationships

- users 1—N documents (uploaded_by)
- users 1—N conversations
- roles 1—N users
- documents 1—N document_chunks
- conversations 1—N messages
- messages 1—N citations
- document_chunks 1—N citations

8.3 Indexing Strategy

- B-tree index on users.email (unique) for fast login lookup.
- Composite index on document_chunks(document_id, chunk_index) for ordered retrieval.
- Index on audit_logs(created_at) to support time-bounded compliance queries.
- Index on messages(conversation_id, created_at) for chronological history retrieval.
- GIN index on documents.metadata (JSONB) for flexible attribute filtering.

8.4 Normalization

The schema is normalized to Third Normal Form (3NF) to eliminate redundant storage of user, document, and conversation attributes. Denormalized read-optimized views are introduced only where analytics dashboards

require aggregate performance, and are refreshed on a schedule rather than stored redundantly in transactional tables.

8.5 Entity-Relationship Summary

```
roles ---< users ---< documents ---< document_chunks ---< citations >--- messages ---<
conversations
users ---< audit_logs
users ---< notifications
```

9. RAG Pipeline

9.1 Document Loader

Format-specific loaders normalize heterogeneous inputs into a common internal representation preserving layout metadata (page, section heading, table boundaries):

Format	Loader Strategy
PDF	Layout-aware extraction (PyMuPDF/pdfplumber) preserving page numbers and table structure; OCR fallback for scanned pages.
DOCX	python-docx traversal preserving headings, tables, and styles for section-aware chunking.
TXT	Encoding-safe text read with paragraph-boundary detection.

9.2 Chunking

Recursive, structure-aware chunking targets 400–800 tokens per chunk with a 10–15% overlap, splitting first on section/heading boundaries, then paragraphs, then sentences, so that no chunk crosses a semantic boundary unnecessarily. Each chunk retains metadata: source document, page, section title, and effective date.

9.3 Embeddings

Chunks and queries are embedded using a bi-encoder sentence-transformer (or an API-based embedding model) into a shared vector space. Embeddings are batched and cached to control latency and cost, and are re-generated whenever the underlying model version changes.

9.4 Vector Database

Vectors are stored with associated metadata in a vector database (pgvector for tight PostgreSQL integration, or a dedicated store such as Qdrant for larger scale), enabling approximate nearest-neighbor search with metadata pre-filtering by document type, role scope, and date range.

9.5 Retriever & Hybrid Search

The retriever combines dense vector similarity with sparse keyword (BM25) search, merging results via reciprocal rank fusion. Hybrid search is critical in this domain because exact terminology, identifiers, and section numbers must be matched precisely, while semantic search captures paraphrased user intent.

9.6 Reranker

A cross-encoder reranker re-scores the top-N hybrid candidates against the exact user query, materially improving precision before the most relevant 4–8 chunks are passed into the LLM's context window.

9.7 Context Injection & Prompt Engineering

Reranked chunks are assembled into a structured context block with explicit source labels. The system prompt instructs the model to answer strictly from the provided context, to state when information is not available, and to attach a citation to every factual claim.

9.8 LLM Response & Citation

The LLM generates an answer grounded in the injected context. The Output Parser cross-checks that every citation in the response maps to an actual retrieved chunk, rejecting or flagging any answer with unverifiable references before it reaches the user.

9.9 Source References

Every answer displays the originating document title, page number, and an excerpt snippet, allowing the user to verify the answer against the primary source in one click — a requirement for any regulated or audit-sensitive workflow.

9.10 Conversation Memory

Short-term memory (the current session's turns) is held in Redis and included in the prompt for coherence. Long-term memory (summarized prior sessions) is persisted in PostgreSQL and selectively retrieved when relevant to the current query.

10. LangChain + LangGraph Flow

10.1 LCEL (LangChain Expression Language)

The pipeline is composed declaratively as a chain of Runnable: prompt | retriever | llm | output_parser. LCEL provides a uniform interface (invoke, batch, stream) across every step, making the pipeline composable, testable in isolation, and easy to reason about.

```
chain = prompt_template | retriever | reranker | llm | output_parser

chain.invoke(query)          # synchronous, single request
chain.batch([q1, q2, q3])    # parallel batch processing
await chain.ainvoke(query)    # async, single request (used in FastAPI routes)
await chain.abatch([...])    # async batch (bulk re-indexing / evaluation jobs)
```

10.2 StateGraph (LangGraph)

For workflows that require more than a single retrieve-then-generate pass, LangGraph models the interaction as a state machine. Each node performs one responsibility (classify intent, retrieve, rerank, generate, validate); edges route the state to the next node based on runtime conditions.

```
START -> classify_intent
classify_intent -> [ retrieve_context | direct_answer | escalate ]    (conditional edge)
retrieve_context -> rerank -> generate_answer
generate_answer -> validate_citations
validate_citations -> [ END | human_approval ]                      (conditional edge)
human_approval -> END
```

10.3 Conditional Edges

Conditional edges inspect the current state (confidence score, document sensitivity flag, citation completeness) and route execution accordingly — for example, sending low-confidence or high-stakes answers to a human-approval node instead of returning them directly to the user.

10.4 Agentic Flow

For multi-step questions (e.g. requiring lookups across multiple document types or a calculation step), LangGraph enables an agent loop where the model can call retrieval or calculation tools iteratively before composing a final answer, with a bounded step count to prevent runaway loops.

10.5 Memory

LangGraph's checkpointing persists the full state of a conversation, enabling pause/resume for human-approval steps and reliable recovery if a node fails mid-execution.

10.6 Fallback Strategy

If the primary LLM provider errors or times out, the graph falls back to a secondary provider or a cached response strategy, and surfaces a clear degraded-mode indicator to the user rather than failing silently.

10.7 Human-in-the-Loop Approval

For high-stakes outputs, LangGraph pauses execution at a `human_approval` node, notifying a reviewer through the Notifications module; the graph resumes only after explicit reviewer sign-off, which is itself recorded in the audit log.

11. Tech Stack

Layer	Technology	Purpose
Frontend	React / Next.js, TypeScript, Tailwind	Chat interface, document management, admin dashboards.
Backend Framework	FastAPI (Python 3.11+)	Async REST API layer.
Orchestration	LangChain (LCEL) + LangGraph	RAG pipeline composition and stateful workflow control.
Primary LLM	OpenAI GPT-4-class models	Answer generation, summarization, structured extraction.
Secondary LLM	Groq-hosted open models	Low-latency fallback and cost-sensitive workloads.
Embedding Model	Sentence-Transformers / OpenAI text-embedding	Dense vector representation of text.
Vector Database	pgvector (or Qdrant for scale)	Semantic similarity search.
Relational Database	PostgreSQL 15+	System of record for users, documents, conversations, audit.
Cache / Session Store	Redis	Caching, rate limiting, session and short-term memory.
Object Storage	S3-compatible blob storage	Durable storage of source documents.
Reverse Proxy	Nginx	TLS termination, load balancing.
Containerization	Docker	Reproducible builds and deployment artifacts.
CI/CD	GitHub Actions	Automated test, build, and deployment pipeline.
Cloud Deployment	AWS / Azure (Kubernetes or ECS)	Managed, autoscaling production hosting.
Observability	OpenTelemetry, Prometheus, Grafana	Metrics, tracing, dashboards, alerting.

12. Production Features

Feature	Implementation Detail
JWT Authentication	Short-lived signed access tokens (15 min) with rotating refresh tokens (7 days), stored httpOnly.
Role-Based Access Control	Declarative permission checks per endpoint, enforced via FastAPI dependencies.
Logging	Structured JSON logs with correlation IDs, shipped to a centralized log store.
Exception Handling	Global exception handlers mapping domain errors to consistent HTTP responses.
Monitoring	OpenTelemetry traces + Prometheus metrics on latency, error rate, and token usage.
Retry Mechanism	Exponential backoff with jitter on LLM/embedding provider calls.
Rate Limiting	Per-user and per-IP token-bucket limits enforced at the gateway and application layer.
Caching	Redis caching of embeddings, frequent queries, and session data.
Pagination	Cursor/offset pagination on every list endpoint with configurable page size.
Search	Hybrid vector + keyword search with metadata filters.
Async APIs	Fully async FastAPI routes and DB access (asyncpg / SQLAlchemy async).
Streaming Response	Server-Sent Events streaming of LLM tokens for perceived low latency.
File Validation	MIME-type, size, and malware-scan checks on every upload.
Security	Encryption in transit (TLS) and at rest, secrets management, input sanitization.
Audit Logs	Immutable, append-only log of every access and administrative action.
Scalability	Stateless API pods behind a load balancer; horizontal autoscaling on CPU/queue depth.

13. Future Enhancements

- Automated regulatory feed ingestion directly from primary regulator publication channels.
- Cross-institution benchmarking of policy language against regulatory best practice (opt-in, anonymized).
- Predictive flagging of policies likely to require review based on regulatory trend analysis.
- Integration with GRC (Governance, Risk & Compliance) platforms for workflow handoff.
- Fine-tuned reranker trained on examiner-validated gap determinations.

14. Risks

14.1 Technical Risks

Risk	Description	Mitigation
Misclassified regulatory scope	A regulation applicable only to certain entity types is incorrectly matched to unrelated policy.	Metadata tagging of applicability scope, validated during ingestion review.
Stale policy corpus	Internal policy updates are not re-ingested promptly, producing false gap alerts.	Automated re-indexing triggers on policy repository change events.
Over-reliance on AI determination	Staff treat AI gap analysis as final without independent review.	Mandatory human sign-off workflow enforced for all high-risk determinations.
Cross-border data residency	Regulatory or policy content subject to jurisdiction-specific storage requirements.	Region-scoped deployment and storage configuration per data residency rules.

14.2 AI Hallucination Risk

Because regulatory gap findings directly inform examiner submissions and remediation commitments, an ungrounded or fabricated answer carries real business and compliance risk. The system mitigates this through strict context-grounding (the model is instructed never to answer outside retrieved context), mandatory citation validation, confidence scoring, and human-approval routing for high-stakes responses.

14.3 Mitigation Summary

- Retrieval-grounded generation with explicit 'answer only from context' instructions.
- Citation-integrity validation before any response reaches the user.
- Confidence thresholds that route uncertain answers to human review.
- Continuous evaluation against a labeled question/answer regression set.
- Versioned prompts and models with rollback capability.

15. Development Roadmap

Phase	Weeks	Deliverables
POC	1-3	Single regulatory domain (e.g. AML) ingestion; core gap-analysis chat; internal demo to compliance leadership.
MVP	4-8	Multi-domain corpus, dual-citation gap detection, RBAC, audit logging, pilot with one compliance team.
Production Hardening	9-11	Security review, penetration testing, monitoring, CI/CD, disaster recovery plan.
Deployment	12	Phased rollout across compliance, audit, and legal functions with examiner-readiness review.

16. Conclusion

Regulatory compliance in banking is a document-intensive, precision-critical domain where a grounded, citation-backed RAG system provides clear, measurable value: faster impact assessment, proactive gap detection, and examiner-ready audit trails.

Given the scale of regulatory change volume and the cost of examination findings or enforcement action, financial institutions have strong incentive to invest beyond this POC into a continuously maintained, production-grade compliance copilot.

Prepared for approval review — AI Mentor sign-off requested prior to POC development kickoff.